

UNIT : 1. : BASICS OF JAVA

Lecture : 1

(*) Introduction

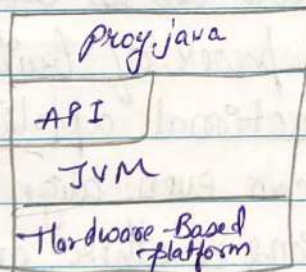
Java was developed by James Gosling in 1991 at Sun Microsystems which is now a subset of Oracle.

The language was initially called as 'Oak', but was renamed as 'JAVA' in 1995.

Java is both programming language & a platform.

Java platform has two components :-

- Java Virtual Machine
- Application Programming Interface



(*) Java Editions.

Java platform provides specialized environments for different categories of applications from small embedded systems to large enterprise servers.

Different Editions of Java are :-

- ① Java Standard Edition (S.E)
- ② Java Enterprise Edition (E.E)
- ③ Java Micro Edition (M.E)
- ④ Java FX.

Java SE is designed for general purpose desktop & server applications.

~~Java~~ Java SE platform includes JVM, JDK, JRE & various class libraries.

It was introduced first to provide core language features & frameworks for general-purpose programming solutions.

Java EE came into ~~picture~~ picture as an ~~extended~~ extension of Java SE with the power of building large-scale, scalable & transactional applications.

Now, Java EE or Jakarta EE can even align with ~~modern~~ modern cloud applications & other enterprise needs.

Java M.E was developed for resource-constrained devices powering mobile & embedded system applications. Java entered these applications & addressed features phones & IoT devices.

Java FX offers a rich UI toolkit while replacing traditional GUI framework like AWT, Swing for desktop applications.

① Components of Java Programming.

To convert the source code to machine code java uses three components:

- ① Java Development kit
- ② Java runtime environment
- ③ Java Virtual Machine.

⇒ Java Development kit (JDK).

It is a software development environment used for making java applications & applets.

It is possible to install more than one JDK version on the same computer.

JDK includes tools required to write java programs & JRE to execute them.

It includes compiler, java application launcher, Appletviewer etc.

⇒ Java Runtime Environment (JRE)

It contains the class libraries, loader class & JVM.

It doesn't include any tool for development. It uses important package classes like math, swing, util, etc.

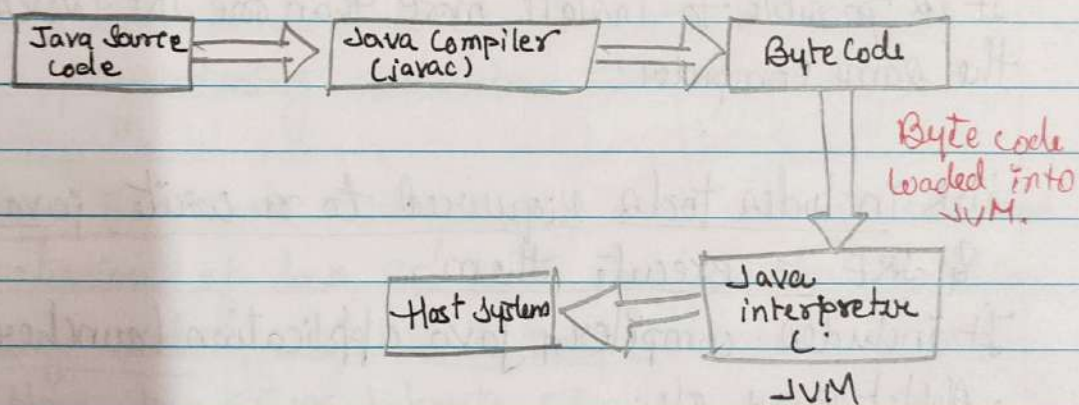
NOTE: For developing the java programs we need JDK but to run java program JRE is needed.

⇒ Java Virtual Machine (JVM)

JVM is an engine that provides a runtime environment to drive a java code or application.

It is a part of JRE which converts bytecode to machine code.

Although JVM provides platform independent way to execute source code but itself is platform dependent.



④ Features of Java.

- ① Simple
- ② Object Oriented programming language
- ③ Distributed
- ④ Robust
- ⑤ Portable
- ⑥ Interpreted & High performance
- ⑦ Architectural Neutral language
- ⑧ Multithreaded
- ⑨ Dynamic

* Java Syntax

```
class Classname{  
    public static void main (String args[])  
    {  
        // statements  
        System.out.println();  
    }  
}
```

Every java program follows a specific structure consisting of classes, methods, statements, keywords & comments.

class is a keyword is used to declare a class, the

Classname :- name of the class. It must start with a capital letter by convention.

public static void main (String[] args).

It is the entry point of a java programme
Execution starts from the main() method.

public :- Accessible from anywhere.

static :- Can be called w/o creating an object

void :- Doesn't return any value

main :- Predefined method name recognized by JVM.

String[] args :- stores command-line arguments.

System.out.println()

Used to display output on the screen.

System is a predefined class. (final class).

out is an output stream object (static variable).

println() prints text & moves the cursor to the next line.

In depth :-

In System class we have.

java.lang package.

```
public final class System {
```

```
    :
```

```
    public final static PrintStream out = null;
```

```
    :
```

```
}
```

In PrintStream class we have.

java.io

```
public void println (String x) {
```

```
    synchronized (this) {
```

```
        print (x)
```

```
        newline ();
```

```
    }
```

```
}
```

hence :- System.out.println();

class

obj of
PrintStream

method
in PrintStream

Program :- First.java [filename can be different]

```
class First {
```

```
    public static void main (String[] args) {  
        System.out.println ("Hello World");  
    }
```

To compile code, we will have to run following commands in terminal.

① `javac First.java`

↳ recommended to be same as the class name. but can be different as well.
filename is

If compiled successfully then,

② `java First`

↳ classname that contains the main method.

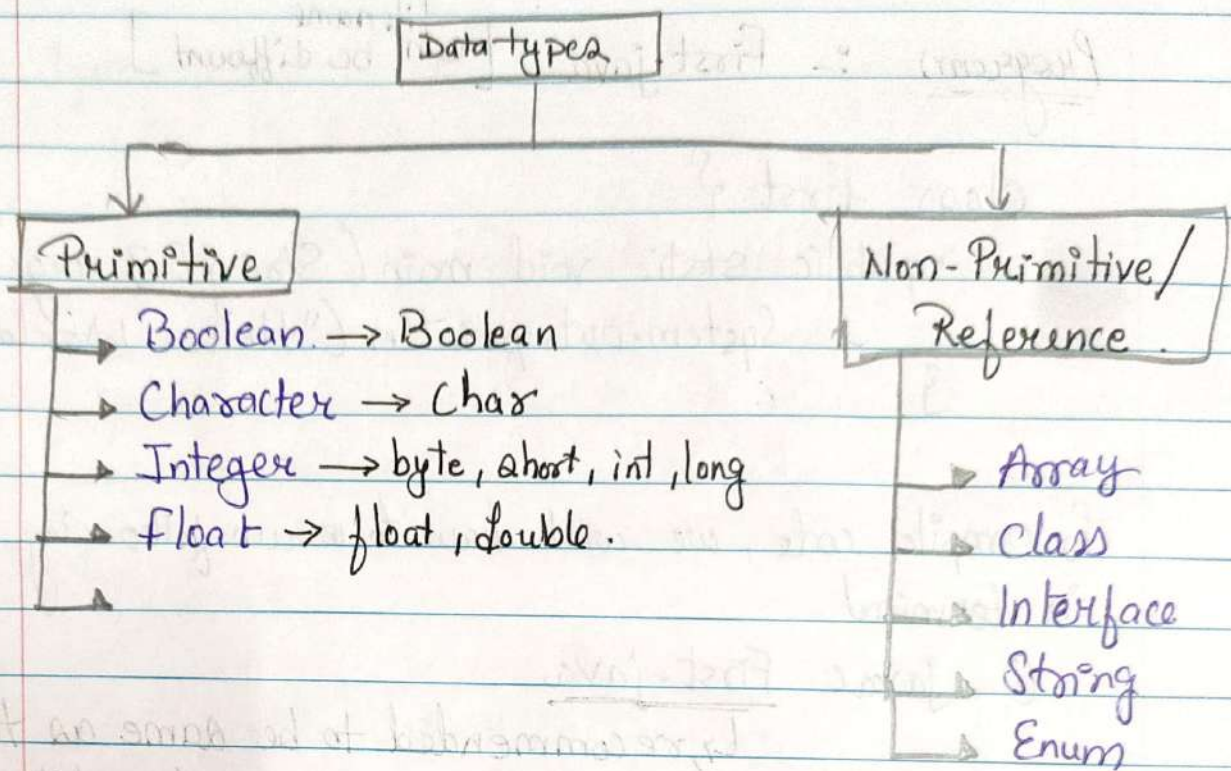
Lecture 2

⑤ Datatypes

These are the specifiers that allocate different size & types of values that can be stored in the variable or an identifier.

Java is a static-typed language.

The type of variable determines its value & operations.



⊛ Variables.

These are the data container that stores the data values during java program execution.

Variable is a memory location name of the data.

Types :-

① Local variables

Variables defined inside methods, constructors or blocks are called local variables.

② Instance variables.

Variable within a class but outside of any method.

③ Static variable

Class variables declared within a class, outside any method with the static keyword.

Eg:-

```

class demo {
    static int a = 1; // static variable
    int b = 10; // Instance variable
    void method () {
        int c = 50; // local variable.
    }
}

```

Rules for variable declaration.

- ① A variable name can consists of capital letter 'A-Z', lowercase letter 'a-z', digits '0-9' & special characters underscore (_) and dollor sign (\$)
- ② First character must not be a digit.
- ③ Java keywords can't be used as variable name
- ④ Blank spaces can't be used in variable name
- ⑤ Variable names are case sensitive
- ⑥ No limit on the length of a variable name, but by convention it should be b/w 4 to 15 chars.
- ⑦ Variable names always should exist on the left-hand side of the assignment.

* User defined Input [Scanner class]

User input in Java allows a program to accept data entered by the user during execution.

The most commonly used class for taking i/p from the keyboard is the `Scanner` class, which belongs to the `java.util` package.

⇒ Syntax :-

```
import java.util.Scanner;
```

```
Scanner sc = new Scanner(System.in);
```

⇒ Reading different datatypes

① `nextLong()` / `nextInt()` → for Integer

Eg :- `int a = input.nextInt();`

② `nextDouble()` / `nextFloat()` → Decimals / Floating-point

③ `nextBoolean()`

④ `next()` → String (a single word [stops at space])

⑤ `nextLine()` → whole line of text (includes spaces)

* Command Line arguments

arguments passing from command prompt.

JVM is responsible for creating `String[]` array of the arguments. & then passing these array arguments in main method.

~~JVM~~ is

Using CLA we can pass any number of arguments & there is no need of declaring variable.

```
Eg:- class Test
    {
        psum (String[] args)
        {
            for (int i=0 ; i<args.length; i++)
            {
                System.out.println (args[i]);
            }
        }
    }
```

Compile: - `javac Test.java` A B C
o `java Test`
o/p: - A B C

Reason for CLA for String

- ↳ In java, most commonly used object is String
 - ↳ It is easy to convert the type of arguments from String
- Eg: - String to number.

Eg2. class Test {

 psvm (String[] args)

 {
 int n = Integer.parseInt (args[0]);

 System.out.println ("The square of " + n + " is " +
 (n*n));

 }
}

converts the String
args to integer.

Compilation :- javac Test.java 10
 java Test

o/p :- 100 The square of 10 is 100.

LECTURE: 3

* Wrapper Class

Used to convert primitive types into class objects. Basically, generic classes only work with objects & don't support primitives.

As a result, if we want to work with them, we have to convert primitive values into wrapper object.

Each primitive type has corresponding wrapper classes.

All wrapper classes belongs to the java.lang package.

Primitive

Wrapper Class

byte

Byte

short

Short

int

Integer

long

Long

float

Float

double

Double

char

Character

boolean

Boolean

Syntax :-

ClassName object = new ClassName(argument);

Eg:- Integer i = new Integer(10);

OR int i = 10;

Integer a = new Integer(i);

valueOf() method : By using valueOf static method, a Wrapper object can be created.

Eg:- `int a = 10;`
`Integer i = Integer.valueOf(a);`

~~Using constructor~~

Using Wrapper class.

Instead of the primitive type to create a wrapper object w/o passing the value to the constructor

Eg:- `Integer a = 10;`
`System.out.println(a);`

Autoboxing

Conversion of primitive value to wrapper object by the compiler is known as autoboxing

Eg:- `Integer i = 10;` // valid as compiler converts int into Integer.

At compile time

`Integer I = Integer.valueOf(10);`

Autounboxing

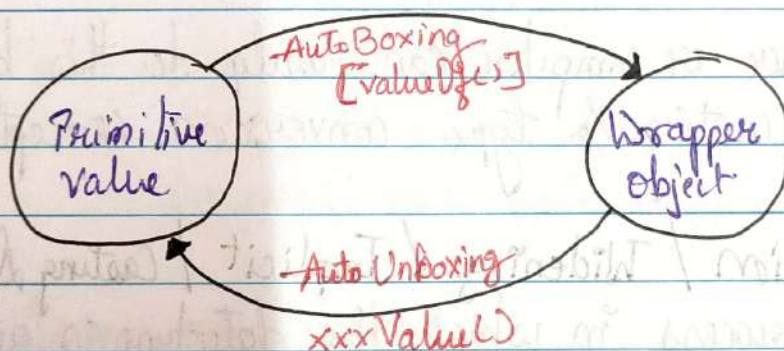
Conversion of wrapper objects to the primitive value by the compiler is called autounboxing

Eg :- `Integer I = new Integer(10);`

`int i = I;`

at compile time

`int i = I.intValue();`



Uses of wrapper class.

↳ Java Collection

↳ Use in generics

↳ null supporter.

i.e. `int x = null` // error

`Integer x = null` // accepted

(*) Type Casting & Type Conversion. NOTE: To be continued after variables

A significant part of programming involves playing around with data.

While performing some tasks, situations might arise where the data type has to be changed from one type to another.

Since, java is ~~strongly~~ ^{often} typed, one ~~must~~ need to convert from one type to another.

The programmer or compiler can easily do this by using type casting & type conversion concepts.

Type Conversion / Widening / Implicit / Casting Down

It is a process in which the datatype is automatically converted into another datatype.

The compiler does this automatic conversion at compile time.

For type conversion to take place, the destination datatype must be larger than source-type

Byte → short → int → long → float → double.

Eg:-
int a = 10;
float b = a;

S.o. `println("Value after type conversion" + b);`

Type casting / Narrowing / Explicit / Casting up.

Process in which the programmer manually converts one datatype into another datatype.

for this the casting operator, "()" is used.

The source datatype must be larger than the destination type.

double $\rightarrow$ float $\rightarrow$ long $\rightarrow$ int $\rightarrow$ short $\rightarrow$ byte.

Eg:- float a = 10.55f;
int b = a;
S.o.println("value after type casting" + b);

⊗ Access modifiers.

These are the keywords used to control the visibility & accessibility of classes, variable, methods & constructors.

They help implement data hiding & encapsulation by restricting unauthorized access to class member.

- ① Public : access level is everywhere. It can be accessed from within the class, outside the class, within package & outside of package.

② Private : The access level is only within the class, it can't be accessed from outside the class

③ Protected : The access level is within the package & outside the package through child class. If you don't make child class, it can't be accessed from outside the package.

④ Default : - The access level is within the package. If you don't specify the access level it will be the default.

* Operators

Operator is a symbol that performs an operation. An operator acts on variables called operands.

Java has following operators.

① Arithmetic Operators $[+, -, *, /, \%]$ ^{Unary} $[-, ++]$

② Relational Operators $[==, !=, <, >, <=, >=]$

③ Bitwise Operators $[&, |, \oplus, \wedge, <<, >>, \sim]$

④ Logical Operators $[&\&, ||, !]$

⑤ Assignment Operator $[+=, -=, *=, /=, \% =]$

⑥ Special operators $[$ Ternary operator $(?:)$
 Instance of $]$

Ternary operator / conditional operator

- Acts on 3 variables.

- Syntax:-

Variable = Expression 1 ? Expression 2 : Expression 3;

- ① Expression 1 is evaluated, if it is true, then Expression 2 is stored in variable.

② If Expression 1 is false, then expression 3 is stored in variable.

instanceOf operator

It is an object reference operator & returns true if the object on the LHS is instance of the class given on the right side.

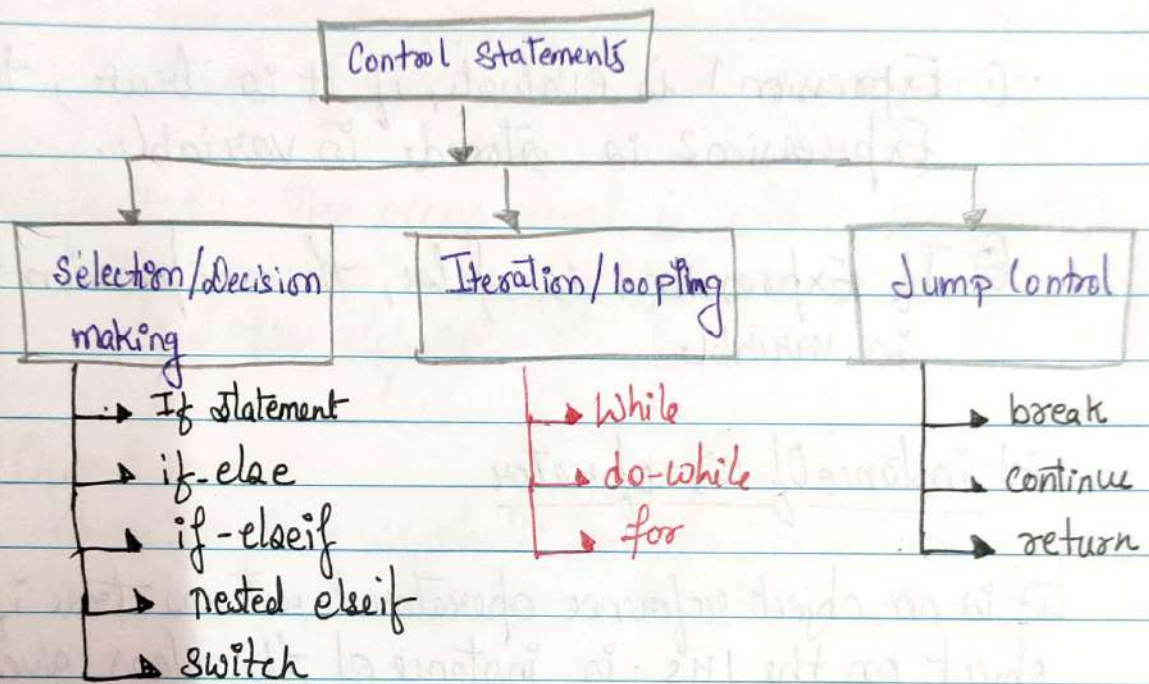
Example :- person instanceof student

If it is true the object person belongs to the class student, otherwise it is false.

LECTURE 5

* Control Statements

These statements alter the flow of execution & provide better control to the programmer on the flow of execution.



⇒ Selection / Decision making

Selection statements allow to choose different paths of execution based upon the outcome of the expression.

Java supports two selection statements:

① if statement

② select statement

① if statement :- performs a task depending on whether a condition is true or false.

syntax :-

<pre>if (condition) { statement; }</pre>
--

here, condition is a boolean expression

condition { → if it is true, statement is executed
→ if it is false, statement is bypassed.

braces `{ }` in `if` are optional, if there is only one statement

if there are multiple statements then it is mandatory to put all the statements of `if` condition within `{ }`.

Eg 1 :- Single statement.

```
if (num < 5)
    System.out.println("number" + num + "is less than 5");
```

Eg 2 :- Multiple statement

```
if (num < 5)
{
    System.out.println("number" + num + "is less than 5");
    System.out.println("This is the multi-statement if condition");
}
```

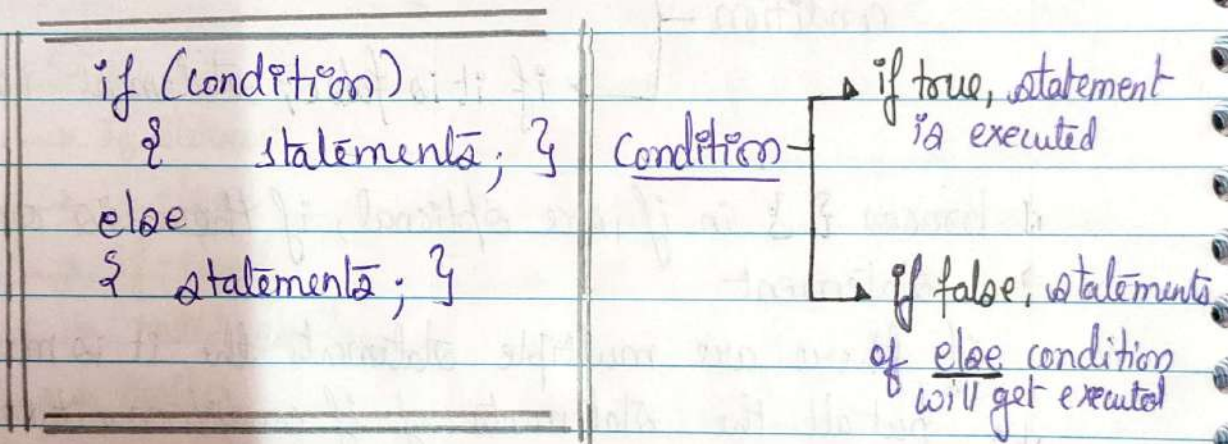
NOTE :- It is good practice to keep the braces with `if`-statement.

There is the drawback of using the if-condition that is if the statement is false it will get bypassed & we won't have any result related to it.

So, the solution is to use the if-else condition.

② if-else condition :

Here syntax is,



Eg:-

```
if (num < 5)
{
    System.out.println("number "+ num + " is less than 5");
}
else {
    System.out.println("number "+ num + " is greater than 5");
}
```

③ if-elseif / elseif ladder

When there are multiple conditions and any one of them can be true, at that time we may use the else-if ladder.

It checks multiple conditions sequentially & executes the corresponding blocks of code for the first condition that is true.

Syntax :-

```
if (condition)
{ statement; }
else if (condition)
{ statement; }
:
:
else { statement; }
```

So let say we have 3 conditions. Condition 1, condition 2 & condition 3, so we will write it like :-

```
if (condition 1)
{ statement; } // if this condition is true, these statement will get executed
else if (condition 2) // if the condition 2 is true, these statement will get executed
{
statement;
}
elseif (condition 3) // if the condition 3 is true, these statements get executed
{
statement;
}
else { // if none of the condition is true, at that time this will get executed
statement;
}
```

In ppt, program need to be updated

④ nested if : In this we have an if condition inside another if condition.

If the inner if condition is satisfied then only outer if will be executed

Along with if-condition we can write else condition also.

Syntax :-

```
if (condition)
{
    if (condition)
    { statement; }
    optional ← else
    { statement; }
}
else { statement; }
```

Example 2 :-

```
if (x < 5)
{
    if (x == 2)
    { s.o.pln("no value of x=2"); }
    else { s.o.pln("value of x=" + x); }
}
```

Example 1:

```
if (x < 5)
{
    if (x == 2)
    { s.o.pln("value of x=2"); }
}
else { s.o.pln("value of x=" + x + "and it is greater than" + x); }
```

```
else { s.o.pln("no. is greater than 5"); } }
```

③ Switch statement

It is a multi-way decision-making statement in java. It allows a program to choose one block of code from several alternatives based on the value of an expression.

It provides a better alternative than a large series of if-else-if statements.

Syntax :

switch (expression)

{ case value 1 :

// statement;

break;

case value 2 :

// statement;

break;

default :

// default statement

}

→ value specified in the case statement

we match it with the expression value, if matches the value, the case value, the statements within that case is executed.

break statement is used to break the case & subsequently breaking the switch statement.

→ If no case matches the expression, then this will get executed.

if not mentioned, all the case statement after the case which matches the expression will get executed.

Example:-

```
int day = 2;  
switch (day)  
{  
    case 1: System.out.println("Monday");  
            break;  
    case 2: System.out.println("Tuesday");  
            break;  
    case 3: System.out.println("Wednesday");  
            break;  
    default: System.out.println("Invalid day");  
}
```

O/p:- Tuesday

NOTE:- We can nest the switch case as well [nested switch]

⇒ Iteration / Looping Statements

Looping statements are used to execute a block of code repeatedly based on a condition.

Instead of writing the same code multiple times, loops allow us to execute a block of code as many times as required based on a condition.

This makes the program clean & more easy.

Suppose you want to print numbers 1 to 100, w/o looping. we will have to write the print code for all numbers from 1 to 100, which will make code too lengthy & messy. and it will become difficult to maintain.

With the loops, the same task is completed with just a few lines of codes.

Loops reduces code repetition, saves time & effort & makes programs shorter.

Java has three looping statements :-

- ① while loop
- ② do-while loop
- ③ for-loop.

① While loop.

while loop repeats a group of statements as long as condition is true. Once the condition is false, the loop is terminated.

Here, first we check for the condition, then only the statements are executed.

⇒ Also called as entry control loop ⇐

Syntax :

```
while (condition)
{
    // statements;
}
```

Condition — If true, body of loop will keep executing
Once get false, control passes to the next line.

Example°.

```
class Whiledemo {  
    public static void main (String args[])  
    {  
        int n=10; // initialization  
        while (n > 0) // condition checking  
        {  
            System.out.println ("value of n=" + n);  
            n--; // updation  
        }  
    }  
}
```

O/p:- value of n = 10

value of n = 9

value of n = 8

value of n = 7

value of n = 6

value of n = 5

value of n = 4

value of n = 3

value of n = 2

value of n = 1

Empty body while loop in Java

The body of the while (or any other loop) can be empty. This is because a null statement is syntactically valid in Java.

Eg:-

```
class EmptyBody {  
    public static void main (String[] args) {  
        int i = 100, j = 200;  
        while (++i < --j); // no body in this loop  
        System.out.println("Midpoint is" + i);  
    }  
}
```

o/p → Midpoint is 150.

Explanation:- value of i is incrementing & value of j is decrementing. At some point the value of $i = \text{value of } j$ & the loop will get break & control will be shifted to next line.

Advantages:-

- ① Makes code shorter
- ② Useful when all processing can be done in the loop condition
- ③ Improves efficiency for simple repetitive operations
- ④ Commonly used by experienced programmers for compact code.

② do-while loop

In the while loop if the initial condition is false, then the body of loop will not executed at all.

However, sometimes it is desirable to execute the body of loop atleast once, even if the condition is false to begin with.

The do-while loop always executes its body atleast once, because its conditional expression is at the bottom of the loop.

Syntax :

```
do {  
    // body of loop  
}
```

while (condition);

→ will execute the body once

→ after that condition will be checked.

Example :

```
class DOWhileLoop {  
    public static void main (String[] args)
```

```
{  
    int i = 1; // initialization  
    do {
```

```
        System.out.println (i + "\t");
```

```
        i++; // updation
```

```
    } while (i <= 10); // condition check
```

if true, loop will be continued.

if false, loop will get terminated.

o/p :- 1 2 3 4 5 6 7 8 9 10

③ for loop

It is useful to execute block of code repeatedly when the number of iteration is known in advanced.

It combines initialization, ~~and~~ condition checking & updation in a single statement, making it compact & easy to use.

Syntax :- for (initialization ; condition ; updation)
{
 // statements
}

Here as well, if condition is true, ~~loop~~ block of code will keep executing. Once condition gets false, control is ~~if~~ been shifted to next line of code.

```
Example:- class ForloopDemo{
    public static void main(String[] args)
    {
        int i;
        for(i=1; i<=10; i++)
        {
            System.out.println(i+"\t");
        }
    }
}
```

Q/p: 1 2 3 4 5 6 7 8 9 10

⇔ Jump Statement

Jump statements are used to transfer control to another part of the program.

Java supports three jump statements :-

- ① break ② continue ③ return.

① break statement

It has three uses :-

- ① Terminates statement sequence in switch
- ② Can be used to exit a loop.
- ③ Using break as a form of goto

⇒ Terminating statement sequence in switch
↳ already seen in switch case.

⇒ To exit a loop:-

- By using break, you can force immediate termination of a loop, bypassing the condition & any remaining code in body of loop.
- When a break statement encounters in loop, the loop is terminated & program control resumes at the next statement following loop.

Example:-

```
class BreakLoop {  
    psvm (String [] args) {  
        for (int i = 1; i < 10; i++) {  
            if (i == 5) { break; }  
            System.out.println (i + "/t");  
        }  
        System.out.println ("Out of loop");  
    }  
}
```

O/p:- 1 2 3 4
Out of loop.

⇒ Using break as a form of goto

Java doesn't have a goto statement because it provides a way to branch in an arbitrary & unstructured manner.

This usually make goto-ridden code hard to understand & to maintain. It also prohibited some compiler optimization

However, there are situations where goto is useful for flow control.

For example:- If we have a deeply nested loop, to exit from it, goto can be useful.

To handle such situation, java defines an extended form of break

This form of break :: work with label, so we can specify precisely where execution will resume.

Syntax : `break label;` → identifies a block of code.

label name :

↳ have to be written before break.

↳ it is an identifier

Example :

```
class BreakLabelDemo
{
    public static void main (String[] args)
    {
        boolean t = true;
```

Label ← `first :` {
 `second :` {
 `third :` {
 System.out.println("Before break");
 if (t) {
 break second;
 }
 System.out.println("This won't get executed");
 }
 System.out.println("This will not get executed as is the second block");
 }
 System.out.println("This will get executed");
 }
}

for second third

o/p:- Before break
This will get executed

We can't break to any label which is not defined for an enclosing block.

Example:-

```
class BlockLabelErr {
    psvm (String[] args)
    {
        one : for (int i=0; i<3; i++)
        {
            System.out.println("i + " + i);
        }
        for (int j=0; j<100; j++)
        {
            if (j==10)
                break one;
            System.out.println("j + " + j);
        }
    }
}
```

must be kept inside the label one. or else will give error

This will give an error.

② Continue

It is used within a loop. It allows you to skip the current iteration of a loop & proceed to the next iteration.

Syntax : `continue;`

Example:-

```
class ContinueDemo {
    psvm (String[] args)
    {
        for (int i=1; i<=10; i++) {
            if (i==5) continue;
            System.out.println (i + " ");
        }
    }
}
```

skip the iteration for i=5.

o/p:- 1 2 3 4 6 7 8 9 10

continue as 'goto' : continue label;

continue may specify a label to describe which enclosing loop to continue.

Eg:- To print a triangular multiplication table for 0 to 5.

```
class ContinueLabelExample {  
    public static void main (String[] args) {  
        Label ← outer : for (int i=0; i<=5; i++) {  
            for (int j=0; j<=5; j++) {  
                if (j>i) {  
                    System.out.println("");  
                    ← enclosing loop ← continue outer;  
                }  
                System.out.println(" " + (i*j));  
            }  
            System.out.println();  
        }  
    }  
}
```

o/p:-

0					
0	1				
0	2	4			
0	3	6	9		
0	4	8	12	16	
0	5	10	15	20	25

③ return;

return statement is useful to terminate a method & come back to calling method.

return statement in main method terminates the application

It can be used to return some value from a method to a calling method.

Syntax:- return;
or return value;

Example:- class ReturnDemo{

public static void main (String[] args){

{ for (int i=0; i<=10; i++)
{ if (i==5) return;

System.out.println(i + " ");

System.out.println("This won't be executed after
encountering the return");

}

o/p:- 1 2 3 4

⊛ Looping ⊛

⇒ foreach ⇐ Also called as enhanced for loop is a special type of for loop introduced in Java to simplify traversing arrays & collection.

It automatically access each element one by one w/o requiring an index variable.

Syntax:-
for (type item : array)
{
 // statements
}

Eg:-
public class ForEachDemo {
 public static void main (String[] args)
 {
 int num[] = {10, 20, 30, 40, 50};
 for (int n : num)
 {
 System.out.println(n + " ");
 }
 }
}

o/p :- 10 20 30 40 50

LECTURE: 7

(*) Arrays

An array is a group of like typed variables that are referred to by a common name.

Arrays of any type can be created & may have one or more dimensions.

A specific element in an array is accessed by its index.

Array provides a convenient means of grouping related information.

Java arrays have :-

- ① Fixed length
- ② Continuous Memory Allocation
- ③ Zero-based Indexing
- ④ Stores primitive & objects
- ⑤ Single Identifier for multiple elements

Declaring array :-

```
datatype [ ] variable-name;
```

Initializing array :-

```
variable-name = { value1, value2, ..., valuen };
```

We can declare & initialize an array in a single step :-

```
datatype [ ] array-name = { value1, value2, ..., valuen };
```

We can create an array using new keyword

' datatype array-name[];

array-name[] = new datatype[N];

number of elements in array

OR
must be same.
datatype[] array-name = new datatype[N]

Eg:-

int[] a = new int[20];

array of 20 elements

Example 1:-

```
class ArrayDemo{
```

```
    public static void main(String[] args){
```

```
        int[] a = {1, 2, 3, 4, 5}
```

```
        for(int i=0; i<a.length; i++)
```

```
        {
            System.out.println(a[i] + " ");
        }
    }
```

```
}
```

o/p :- 1 2 3 4 5

a.length specifies the length of array 'a' i.e. 5

NOTE:-

Once the length of the array is specified, we can't directly add more elements.

Eg:- int[] a = {1, 2, 3, 4}

a[4] = 5 // gives error.

& initialized

Changing value of an element in array.

```
int[] a = {10, 20, 30, 40, 50};
```

```
a[3] = 70;
```

→ it will change the value of element in index 3
i.e. 40 to 70.

To find the length of an existing array we can use length keyword

```
int[] a = {10, 20, 30, 40, 50};
```

```
System.out.println(a.length);
```

↓
gives o/p as 5.

Remember, length is the total no. of elements in an array.

Index is the position number of an element in an array & it starts from 0.

index $\neq$ length.

We can specify or access the array element values using the loops [mostly used is for loop]

Example:-

```
import java.util.Scanner;  
class ArrayLoop{  
    psvm (String[] args)
```

```
    Scanner sc = new Scanner(System.in);
```

```
    System.out.println("Enter number :- ");  
    int a = new int[5];  
    for (int i : a)  
    {  
        i = sc.nextInt();
```

```
    }  
    System.out.println("Number entered are :- ");  
    for (int i : a)  
    {  
        System.out.println(i + " ");  
    }
```

Output :- Enter number :-

1

2

3

4

5

Number Entered are :-

1

2

3

4

5

2

3

⇒ Multi-dimensional Arrays

Arrays of arrays are called multidimensional arrays.

To declare a multidimensional array variable, specify each additional index using another set of square bracket-

declaration :-

```
[datatype][ ][ ] arrayname;
```

Example :-

```
class Arr {
```

```
    sum (String[ ] args)
```

~~of~~

```
{
```

```
int a[ ][ ] = new  
int[3][3]
```

if we want to
take input from
user.

```
int a[ ][ ] = { {1, 2, 3}, {4, 5, 6} };
```

```
for (int i : a) {
```

```
    for (int j : i) {
```

```
        System.out.print(a[i][j] + " ");
```

```
    }  
    System.out.println(" ");
```

3 { }
Output :-

1 2 3

4 5 6

=> Jagged array

It is the multidimensional arrays with different size. [i.e. each array in ^{each} column are of different size].

Example:-

```
class JaggedArray {  
    > sum (String[] args) {
```

```
        int a[][] = { {1, 2}, {3, 4, 5},  
                       {6, 7, 8} };
```

```
        for (int i : a) {
```

```
            for (int j : i) {
```

```
                System.out.print(a[i][j] + " ");
```

```
            }
```

```
            System.out.println(" ");
```

```
        }
```

```
    }
```

```
}
```

Output:-

1 2

3 4 5

6 7 8

⊗ Strings

A string in java is a sequence of characters used to store & manipulate text.

String is a predefined class available in the java.lang package..

As java.lang package is automatically imported, there is no need to explicitly import the String class.

NOTE:- Strings are immutable i.e we can't change string once it is created.

Any operation that is use to modify a string actually creates a new String object

Creating Strings :-

String can be created in two ways:-

① Using String Literal:-

```
String name = "Java";
```

② Using the 'new' keyword:-

```
String name = new String("Java");
```

⇒ String Methods

① length()

- Returns the length of the string
- It will calculate all the characters, symbols & numbers enclosed in double quotes (" ").

Syntax:-

```
StringName.length();
```

Example:- String a = "hello";

S.O.pln("Length of string : " + a.length());

O/P : Length of string : 5

② toLowerCase()

Convert all upper case letters of the string to lowercase

Syntax:-

```
StringName.toLowerCase();
```

③ toUpperCase() : Convert all lowercase letters of string to uppercase.

Syntax:-

```
StringName.toUpperCase();
```

④ substring() : Returns a new String instance from the original string.

Syntax:-

```
StringName.substring(start, end);  
OR  
StringName.substring(start);
```

⑤ concat() : concatenate the strings.

Syntax :-

```
stringName.concat(stringName2);
```

⑥ equals() & equalsIgnoreCase()

↳ compare strings while ignoring character case.

↓
→ Compared two strings

Syntax :-

```
stringName1.equals(stringName2);  
stringName1.equalsIgnoreCase(stringName2);
```

⑦ indexOf() :- Returns the position of a character or substring

Syntax :-

```
stringName.indexOf('a');
```

⑧ contains() : Check whether a string contains specific text.

Syntax :-

```
stringName.contains('a');
```

⑨ replace() :- Replaces characters or substrings

Syntax :-

```
stringName.replace('old character', 'new character');
```

Example:-

```
class StringOperation {
```

```
    public static void main (String[] args) {
```

```
        String s = "Hello There";
```

```
        String str1 = "hello there";
```

```
        String str = ", how are you";
```

```
        System.out.println(s.length());
```

```
        System.out.println(s.toLowerCase());
```

```
        System.out.println(s.toUpperCase());
```

```
        System.out.println(s.substring(6));
```

```
        System.out.println(s.concat(str));
```

```
        System.out.println(s.equals(str1));
```

```
        System.out.println(s.equalsIgnoreCase(str1));
```

```
        System.out.println(s.indexOf('o'));
```

```
        System.out.println(s.contains('e'));
```

```
        System.out.println(s.replace('Hello', 'Hi'));
```

```
    }
```

```
}
```

Output:-

* String Buffer

As we know, String is of fixed-length, immutable, so it leads to consume extra memory & reduces performance when many modifications are performed.

StringBuffer allows modifications to the same object without creating new objects repeatedly.

Like String, StringBuffer also belongs to java.lang package.

StringBuffer can be defined as below.

- ① StringBuffer sb = new StringBuffer();
- ② StringBuffer sb = new StringBuffer(int size);
- ③ StringBuffer sb = new StringBuffer(String str);

The StringBuffer(String str) creates an object whose initial contents are same as the specified string. In addition to storing the given string, Java automatically reserves extra space for 16 additional characters.

Eg:- StringBuffer sb = new StringBuffer("Java");
System.out.println("Capacity: " + sb.capacity());

o/p capacity:- 20
(Java + 16 additional)

⇒ StringBuffer Methods

① length() :- To get the current length of the object

Syntax :- `StringBufferObject.length()`

② capacity() :- Total allocated capacity can be found through capacity method.

Syntax :- `SbObject.capacity()`

③ charAt() & setCharAt()

charAt() :- To obtain value of a single character

setCharAt() :- To set the value of a character

Syntax :- `Sb.charAt(int index)`

`Sb.setCharAt(int index, char ch)`

④ append() :- Concatenates the string representation of any other type of data to the end of the invoking StringBuffer object

Syntax :- `Sb.append(String str)`
`Sb.append(int num)`
`Sb.append(Object obj)`

⑤ insert() :- Inserts one string into another.

Syntax:-

```
sb.insert(int index, String str)
sb.insert(int index, char ch)
sb.insert(int index, Object obj)
```

⑥ reverse() :- To reverse the characters within an object.

Syntax:-

```
sb.reverse()
```

⑦ delete() & deleteCharAt().

delete() :- deletes a sequence of characters from the invoking object.

Syntax:-

```
sb.delete(int startIndex, int endIndex)
```

deleteCharAt() :- deletes the character at the specific index

Syntax:-

```
sb.deleteCharAt(int index)
```

⑧ replace() :- Replaces one set of characters with another set

Syntax:-

```
sb.replace(int startIndex, int EndIndex,
           String str)
```

⑨ substring() :- To get a part of string from the invoking object.

Syntax:-

sb.substring(int startIndex)

sb.substring(int startIndex, int endIndex)

Example :-

```
class StringBufferExample{
```

```
    public static void main (String[] args){
```

```
        StringBuffer sb = new StringBuffer("Hello");
```

```
        System.out.println("Length of String: " + sb.length());
```

```
        System.out.println("Capacity of String: " + sb.capacity());
```

```
        System.out.println("Character at index 2: " + sb.charAt(2));
```

```
        System.out.println("Set Character at index 5: " + sb.setCharAt(5, 'i'));
```

```
        System.out.println("Set Appended String: " + sb.append("There"));
```

```
        System.out.println("Inserted character: " + sb.insert(2, "o"));
```

```
        System.out.println("Reversed String: " + sb.reverse());
```

```
        System.out.println("Deleting String: " + sb.delete(5, 9));
```

```
        System.out.println("String after deleting char. at specific index: " +  
                           sb.deleteCharAt(4));
```

```
        System.out.println("String after replace: " + sb.replace(1, 4, "i"));
```

```
        System.out.println("Substring is: " + sb.substring(0, 3));
```

```
    }
```

```
}
```